Morning Brief

Ludovici · Saturday, June 13, 2026

In this edition

By Stephanie Ludovici

A personal learning and awareness brief: AI governance, cybersecurity, project status, five new project ideas, and four training items. Built to keep me current and sharp.

AI governance & policy

- **NIST publishes IR 8596, a Cybersecurity Framework Profile for AI (Feb 2026).** It extends CSF 2.0 to AI-specific risk, organized around Secure / Defend / Thwart — a direct bridge between your framework's security-evaluation dimension (§06) and an authoritative cyber baseline. NIST AI standards
- **NIST releases a concept note for an AI RMF Profile on Trustworthy AI in Critical Infrastructure (Apr 7, 2026).** A sector profile is exactly the pattern a tiered IC framework can borrow from for mission-context tailoring. NIST AI RMF
- **Singapore's IMDA Model AI Governance Framework for Agentic AI is now circulating with security guidance (launched at WEF, expanded Apr 2026).** A second national-level anchor for agentic governance — useful triangulation beyond OWASP, which your v3.0 tracker flags as the lone surviving source for the agentic overlay. IMDA agentic framework

- **WEF publishes "Making Agentic AI Work for Government: A Readiness Framework" (2026).** A government-specific readiness lens for agents that plan and act — relevant comparator for your Phase-5 agent-evaluation idea. WEF readiness framework
- **EU AI Act: high-risk obligations become fully applicable August 2, 2026.** The near-term date keeps sharpening the global high-risk-classification baseline against which to benchmark your tier definitions. EU AI Act framework

Cybersecurity & threats

- **Microsoft June Patch Tuesday sets a record: ~208 CVEs, including a wormable Windows kernel flaw (CVE-2026-45657, CVSS 9.8).** EoP (~32%) and RCE (~27%) dominate. Federal-estate priority: kernel, HTTP.sys/IIS, RDS, Hyper-V, and BitLocker-protected devices. SecurityWeek · Tenable
- **CISA rolls out a new patch-prioritization process** (per SANS NewsBites Jun 12) — a shift toward dynamic, continuous prioritization rather than fixed BOD deadlines. Worth tracking for how it reshapes the federal patch SLA model. CISA advisories
- **CISA adds three KEVs (Jun 9):** CVE-2026-7473 (Arista EOS incomplete comparison), CVE-2026-11645 (Chromium V8 OOB read/write), CVE-2026-20245 (Cisco Catalyst SD-WAN Manager output-encoding). The Chromium and Cisco items have broad federal reach. CISA KEV add, Jun 9
- **CISA issues nine ICS advisories (Jun 11)** spanning Siemens, Schneider Electric, Rockwell Automation, Mitsubishi Electric, Advantech, National Instruments, Inductive Automation, and Axis — OT/critical-infrastructure exposure with clear federal relevance. Industrial Cyber summary
- **CISA + allied partners issue a joint advisory on Chinese state-sponsored covert cyber networks** feeding global espionage — squarely within the IC threat model. CISA news

Project status

- **NetMon-Server** *(most active)* — Single-file, loopback-only network monitor / light SIEM with an optional on-device (Ollama) AI layer; docs are mid-rewrite (`DOC-REWRITE-BRIEF.md`,

`TECHNICAL-MANUAL-v0.25.0`). **Next action:** with this week's record Patch Tuesday and CISA's new prioritization model, add a detection/alert rule (or posture-score note) that surfaces unpatched high-severity KEV-class exposure on the host.

- **MongoDB (vetting-pipeline discovery)** — Discovery runbook and illustrated process paper for a vetting data pipeline, now in polished docx/pdf form with DFDs. **Next action:** capture a short data-lineage diagram for the pipeline (ties directly to today's data-centric training item).
- **AI Framework (v2.x shipped; v3.0 tracked)** — `V3.0-ROADMAP-FOLLOWUP.md` keeps three deferred overlays alive: categorical pre-screen (prototyped), agentic governance (OWASP single-source), and GenAI groundedness. **Next action:** log today's IMDA agentic framework and WEF government readiness framework into the agentic-overlay evidence base — both help triangulate beyond OWASP, the exact gap the tracker calls out.

Five project ideas

Full write-ups (with problem statement, objective, method, duration, sources) saved to `project-ideas/2026-06-13-ideas.md`. Summary:

1. **Persistent homology (TDA) for nanoporous-material characterization** — Materials science · Python (`giotto-tda`/`gudhi`) + R (`TDA`). *Problem:* scalar pore descriptors discard void-space connectivity. *Objective:* test topological descriptors against geometric baselines for adsorption prediction. *Method:* persistence diagrams → vectorization → boosted/kernel regressor, nested CV. *Duration:* 3–4 weeks.
2. **Spiking neural networks for event-camera classification** — Neuromorphic / neuro-inspired · Python (`snnTorch`/`Norse`). *Problem:* CNNs handle sparse asynchronous spikes poorly. *Objective:* quantify the accuracy-vs-energy-proxy trade-off vs a frame CNN. *Method:* LIF SNN with surrogate-gradient training; ablate time steps. *Duration:* 3–5 weeks.
3. **Hawkes/ETAS point process for aftershock forecasting** — Seismology · R (`PtProcess`) + Python (`tick`). *Problem:* independence assumptions underestimate short-term hazard. *Objective:* short-horizon forecast skill vs a Poisson baseline. *Method:* ML estimation of the ETAS conditional intensity, time-

rescaling residuals, information-gain evaluation. *Duration:* 2–3 weeks.

4. **State-space models (Mamba/S4) for climate teleconnection forecasting** — Climate · Python (PyTorch). *Problem:* long-range dependencies are awkward for RNNs and costly for full attention. *Objective:* multi-month index forecasting vs LSTM/transformer on RMSE, anomaly correlation, and compute. *Method:* train SSM + baselines on decades of public index data. *Duration:* 4–6 weeks.
5. **Optimal transport for single-cell trajectory inference** — Genomics · Julia (`OptimalTransport.jl`) + Python (`POT`). *Problem:* scRNA-seq gives distribution snapshots, not trajectories. *Objective:* reconstruct developmental couplings and validate against lineage markers. *Method:* Sinkhorn transport plans between time points, chained couplings, held-out interpolation error. *Duration:* 3–5 weeks.

Training

1. Python — `functools.lru_cache` / `cache` for memoization
Memoization caches a function's return value keyed by its arguments, so repeated calls with the same inputs are answered instantly. `functools` makes this a one-line decorator. It matters because it turns exponential recomputation (recursive DP, repeated I/O-free lookups) into linear work without restructuring your code.

```
# Demonstrate memoization on a naively exponential recurs
import functools
import time


@functools.lru_cache(maxsize=None)  # Unbounded cache; us
def fib(n):
    # Each value is computed once, then served from the c
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)


start = time.perf_counter()
value = fib(100)  # Instant with caching; intractable wit
elapsed = time.perf_counter() - start  # Two spaces befor
```

```
print(f"fib(100) = {value} in {elapsed * 1e6:.1f} microse
print(fib.cache_info())  # Inspect hits, misses, and curr
```

Common pitfall: arguments must be hashable, so lists and dicts will raise `TypeError` — pass tuples or frozensets instead. And `maxsize=None` grows without bound, which can leak memory in a long-running process; cap it (e.g., `maxsize=1024`) or call `.cache_clear()` when the underlying data changes, or stale results will silently persist.

Reference: Python docs, `functools` — https://docs.python.org/3/library/functools.html

2. R — `dplyr::across()` for column-wise operations

`across()` applies one or more functions to a tidy selection of columns inside `summarise()` or `mutate()`, replacing the old `summarise_at`/`_if`/`_all` family. It matters because it keeps group-wise aggregation concise and readable while scaling to many columns without copy-paste.

```
library(dplyr)

# Summarize every numeric measurement by species in one e
iris |>
  group_by(Species) |>
  summarise(
    across(
      where(is.numeric),                # Tidy-select: c
      list(mean = mean, sd = sd),       # Apply multiple
      .names = "{.col}_{.fn}"           # Control output
    ),
    n = n(),
    .groups = "drop"
  )
```

This returns one row per species with `*_mean` and `*_sd` columns plus a count. Idiomatic modern R favors the native `|>` pipe and `across()` over repetitive per-column code.

Common pitfall: forgetting `.groups = "drop"` leaves the result grouped, so the next verb silently operates within groups. Be explicit about grouping state.

Reference: dplyr `across()` — https://dplyr.tidyverse.org/reference/across.html

3. Data-centric — Data lineage and provenance

Lineage records where data came from, how it was transformed, and where it flows — a column- and dataset-level map from source to

consumption. It matters for trust and governance: when a number looks wrong, lineage lets you trace it to the offending transform; when a source is retired or found to be poisoned, lineage tells you every downstream artifact that must be revalidated. For AI/IC work it is also a control surface — provenance of training and retrieval data is what lets you reason about integrity, classification flow-down, and confabulation risk.

A minimal lineage record per dataset/version:

```
dataset: vetting_subjects_curated
version: 2026-06-13
sources:
  - name: case_management_export
    system: CMS
    extracted_at: 2026-06-12T23:00Z
transformations:
  - step: deduplicate_on_subject_id
    code_ref: pipeline/dedupe.py@a1b2c3d   # Pin the exac
  - step: redact_pii_fields
    fields: [ssn, dob]
consumers:
  - adjudication_scoring_model
  - leadership_dashboard
```

Captured this way (whether by hand, OpenLineage, or a catalog), lineage turns "who touched this?" from an investigation into a lookup — and supports the integrity controls your governance framework expects.

Reference: OpenLineage — https://openlineage.io/docs/

4. AI method — Gaussian Process Regression

A Gaussian Process (GP) defines a distribution over functions: any finite set of points is jointly Gaussian, with covariance given by a kernel that encodes how strongly nearby inputs co-vary. Conditioning that prior on observed data yields a posterior with a closed-form mean **and** variance at every test input. The intuition: instead of fitting one curve, you keep the whole family of curves consistent with the data and report both their average and their spread.

When to use it: small-to-moderate datasets where calibrated uncertainty matters — surrogate modeling, active learning, and especially **Bayesian optimization** of expensive experiments or simulations. The headline cost is the dense kernel solve, which scales as $O(n^3)$; beyond a few thousand points you move to sparse/inducing-point approximations. The companion notebook for this edition —

`notebooks/2026-06-13-gaussian-process-regression.ipynb` — implements GP regression from scratch (NumPy + matplotlib), with explicit **Modeling & Simulation** (data drawn from a known function) and **Test & Evaluation** (held-out RMSE plus 95% predictive-interval coverage) sections.

Reference: Rasmussen & Williams, *Gaussian Processes for Machine Learning*, MIT Press (2006) — https://gaussianprocess.org/gpml/

Note: this brief discusses cybersecurity threats and vulnerabilities for defensive awareness only.

Ledgers updated: five idea titles appended to `project-ideas/_ledger.md`; four training topics appended to `training-ledger.md`.

Attached: brief (.pdf) + companion notebook (.ipynb) + project briefs (.docx)
Morning Brief · Ludovici · Saturday, June 13, 2026
By Stephanie Ludovici